

ANÁLISIS DEL RETO

Danna Jael Alape Mosquera, 202410457, d.alape@uniandes.edu.co

Nicolas Rene Sarmiento Melo, 202524791, nr.sarmiento@uniandes.edu.co

Juan José Castro Maldonado, 202520030, j.castrom23@uniandes.edu.co

Requerimiento 1

```
def req_1(catalog, modelo, precio_min, precio_max):  
    """  
    Retorna el resultado del requerimiento 1  
    """  
    # TODO: Modificar el requerimiento 1  
  
    start = get_time()  
    modelo = modelo.lower().strip()  
    map_req1 = catalog["req1"]  
    arbol_modelo = lp.get(map_req1, modelo)  
  
    if arbol_modelo is None:  
        return {"total": 0, "promedio": 0, "resultados": al.new_list(), "tiempo": delta_time(start, get_time())}  
  
    precio_min_val = float(precio_min)  
    precio_min_str = format(precio_min_val, "010.2f")  
  
    llave_min = precio_min_str + "-0"
```

Descripción

El requerimiento busca, dado un modelo de vehículo un rango de precios, (min, max), retornar todas las ventas que cumplan ambos filtros. La implementación usa un mapa hash que asocia cada modelo con un árbol RBT cuyos nodos están ordenados por precio, permitiendo búsquedas por rango en $O(\log n+k)$.

Una vez obtenidos los k resultados del árbol, se convierten en un ArrayList y se ordenan con merge sort según el criterio: precio ascendente, luego horsepower descendente en caso de empate, y luego color alfabético. Si el total supera 12, se imprimen los 6 primeros y los 6 últimos.

Entrada	Modelo, precio_min, precio_max
Salidas	Total de ventas en el rango, promedio de precio y lista de ventas ordenadas por precio ascendente
Implementado (Sí/No)	Si, Implementado por Nicolás Sarmiento

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Buscar el arbol del modelo en el mapa hash	$O(1)$
Búsqueda por rango en el RBT	$O(\log n + k)$
Recorrer los k resultados para calcular la suma de precios	$O(k)$
Convertir el <code>single_linked_list</code> a <code>array_list</code>	$O(k)$
Ordenar los k resultados con <code>merge_sort</code>	$O(k \log k)$
Seleccionar índices a mostrar	$O(1)$
TOTAL	$O(\log n + k \log k)$

Análisis

La operación dominante es el merge sort sobre los k resultados del rango ($O(k \log k)$), dado que la búsqueda en el RBT ya es eficiente ($O(\log n + k)$). En el peor caso, si todos los registros de un modelo caen dentro del rango, $k = n_{\text{modelo}}$ y la complejidad total sería $O(n \log n)$. En la práctica, los rangos de precio suelen acortar k considerablemente, haciendo el requerimiento muy rápido.

# de prueba	Datos de entrada	Tiempo de ejecución
1	Modelo =GLC, precio min=70000, precio max=10000	207.81 ms
2	Modelo = GLA, precio min=30000, precio max=200000	389.48 ms
3	Modelo =GLS, precio min=70000, precio max=10000	129.52 ms
4	Modelo = GLE, precio min= 50000, precio max=150000	154.74 ms
5	Modelo = S-Class, precio min=60000, precio max=500000	113.72 ms
Promedio de ejecución		213.766 ms

Requerimiento 2

```
def req_2(catalog, combustible, minimo, maximo):  
    """  
    Retorna el resultado del requerimiento 2  
    """  
    # TODO: Modificar el requerimiento 2  
    start_time = get_time()  
  
    map_req2 = catalog["req_2"]  
    arbol_req2 = lp.get(map_req2, combustible.lower().strip())  
  
    unidad_vendida = 0  
    suma_precios = 0  
    suma_horsepower = 0  
    resultado = al.new_list()
```

Descripción

Este requerimiento realiza una búsqueda filtrando en base a el tipo de combustible y un rango con Horsepower. Utilizando una estructura de Mapa de Hash para el acceso directo por combustible en la carga de datos y un arbol rojo-negro para realiza el filtrado por rango, la función se encarga de identificar los carros que cumplen con los filtros para calcular el total de unidades vendidas, precio promedio y el promedio de los caballos de fuerza. La función retorna un diccionario debido a la cantidad de información que debe dar para facilitar la salida de los datos en el view.

Entrada	Catálogo, tipo de combustible, minimo y maximo de horsepower para crear el rango
Salidas	Diccionario con tiempo de ejecucion, total de carros que cumplen con el filtro, promedio de precios y de horsepower y una lista con los primeros y ultimos 6 carros vendidos que cumplen con el filtro
Implementado (Sí/No)	Sí, implementado por Danna Alape

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Inicialización de variables - get_time(), limpieza de string, concatenación de llave, acceso a mapa y a sublista con lp.get()	$O(1)$
Acceso a llaves del árbol - key_set sobre el RBT	$O(m)$, donde m es la cantidad de carros con el combustible pedido
Filtrado por rango y obtención de datos - Ciclo for para acceder a cada llave con rbt.get() y calculo de promedios	$O(m \log m)$
TOTAL	$O(m \log m)$

Análisis

La eficiencia y velocidad de este requerimiento se debe al uso de las Tablas de Hash y los RBT. Se utiliza un mapa que recibe por llave el tipo de combustible, lo que permite que el tiempo de ejecución no

dependa del tamaño total del catálogo (N), sino que dependa de la cantidad de carros que cumplen con el filtro solicitado (m). Al utilizar la entrada del usuario como llave, el algoritmo tiene un acceso de tiempo $O(1)$ al árbol que contiene los datos. Aunque se debe realizar un recorrido al árbol para filtrar el rango, esta operación se realiza en una sublista donde $m < N$. En conclusión, esta estructura y uso de las llaves garantiza un rendimiento eficaz y escalable, ya que evita realizar recorridos y revisar todos los carros.

# de prueba	Datos de entrada	Tiempo de ejecución
1	Combustible =Hybrid, min_HP=500, max_HP = 600	568.29
2	Combustible =Hybrid, min_HP=500, max_HP =600	857.9
3	Combustible =Gasoline, min_HP=300, max_HP =450	682.4
4	Combustible =Diesel, min_HP=420, max_HP = 250	1156.5
5	Combustible =Plug-in Hybrid, min_HP=400, max_HP = 550	976.7
Promedio de ejecución		848.36

Requerimiento 3

Plantilla para documentar y analizar cada uno de los requerimientos.

Descripción

Para este requerimiento se utilizó una combinación entre tabla de hash LP y RBT. Primero se accede directamente a el árbol asociado al año y tipo de combustible que se solicita por parámetro dentro de la tabla de hash, garantizando $O(1)$. Después se recorren unicamente las ventas que cumplen la condición, y se filtran las que entren en el rango de precios. Los resultados se insertan en un nuevo RBT, respetando el orden que pide el requerimiento: Precio base descendente y Horsepower descendente. Finalmente, se recorren los resultados ya ordenados y se calcula el promedio de precios.

Entrada	Año, tipo de combustible, precio minimo, precio maximo
Salidas	Tiempo de ejecución en milisegundos, número total de ventas encontradas, promedio de precio base de las ventas filtradas y lista de ventas que cumplen la condición: model, year, fuel type, color, base price (USD), horsepower, turbo
Implementado (Sí/No)	Si se implementó por Juan José Castro

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Acceso a mapa con año y fuel type	$O(1)$
Obtener llaves del árbol asociado	$O(n)$
Recorrido y filtro de ventas	$O(n)$
Inserción de resultados al RBT	$O(k \log k)$
Recorrido de resultados ordenados	$O(k)$
TOTAL	$O(n + k \log k)$

Análisis

Gracias al uso de las tablas de hash se evita recorrer la totalidad de los datos cargados y se puede trabajar en la parte que nos indican (año y tipo de combustible específico). El uso de RBT nos permite mantener un orden sin la necesidad de aplicar ningún algoritmo de ordenamiento adicional. La complejidad $O(n + k \log k)$ se debe a la inserción de los resultados ya filtrados en el RBT. El número de elementos filtrados es menor al total de elementos cargados, lo que asegura la eficiencia del requerimiento. Se evitan recorridos innecesarios en los datos que no cumplen las condiciones del requerimiento.

Procesador: 1,4 GHz Intel Core i5 de cuatro núcleos

# de prueba	Datos de entrada	Tiempo de ejecución
1	2023, Hybrid, 60000, 120000	857.96 ms
2	2024, Petrol, 40000, 100000	1184.72 ms
3	2022, Diesel, 50000, 100000	682.48 ms
4	2025, Hybrid, 70000, 160000	1156.58 ms
5	2021, Petrol, 50000, 150000	976.71 ms
Promedio de ejecución		971.69 ms

Requerimiento 4

```
def req_4(catalog, anio, N):
    """
    Retorna el resultado del requerimiento 4:
    Para un año dado, identificar los N modelos con mayor número de vehículos vendidos.
    """

    start = get_time()

    reporte = catalog["reporte_carga"]
    llave_min = f"{int(anio):04d}-0000000000.00-"
    llave_max = f"{int(anio):04d}-9999999999.99~"
    carros_anio = rbt.values(reporte, llave_min, llave_max)
```

Descripción

Para un año dado, el requerimiento identifica los N modelos con mayor volumen de ventas, reportando para cada uno el precio promedio, horsepower promedio, porcentaje de turbo y el vehículo de mayor horsepower. Se usa el reporte carga (RBT global) para extraer todos los carros del año con una búsqueda por rango de llaves con formato '{año}-precio-modelo-id', y luego se agrupa por modelo en un mapa de hash temporal.

Una vez agrupados, se construye un ArrayList con las estadísticas de cada modelo y se ordena descendientemente por ventas (empate: precio promedio descendente). Se toma el sublist de los primeros N. El vehículo de mayor HP por modelo se actualiza en línea durante el recorrido, comparando HP y aplicando los desempates de menor precio y años mas antiguo.

Entrada	Catalogo, año, N (cantidad de modelos que se quieren identificar)
Salidas	Total de modelos en el año, y Top N modelos con: Ventas, precio promedio, HP promedio, % turbo y venta de mayor HP.
Implementado (Sí/No)	Si, Implementado 45% Nicolas, 25% Danna, 30% Juan

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Búsqueda por rango del año en el RBT global	$O(\log N + k)$
Recorrer k carros del año y agrupar por modelo en mapa hash temporal	$O(k)$
Iterar la tabla del mapa para construir ArrayList de modelos	$O(M_capacity)$
Ordenar M modelos con merge sort	$O(M \log M)$
Tomar sublist de los primeros N modelos	$O(N)$
TOTAL	$O(\log N + k + M \log M)$

Análisis

La operación de búsqueda por rango en el RBT global extrae únicamente los k carros del año solicitado sin recorrer todo el dataset, lo que es considerablemente más eficiente que un filtro lineal $O(n_total)$. El agrupamiento por modelo en el mapa hash temporal garantiza $O(1)$ amortizado por inserción, haciendo que el recorrido de los k carros sea $O(k)$ en total. El término dominante en la práctica es $O(k)$, ya que M (número de modelos distintos) es mucho menor que k, convirtiendo el sort de modelos en un costo casi constante. La iteración de la tabla interna del mapa es $O(M_capacity) = O(10.000)$, que también es constante para cualquier año consultado.

# de prueba	Datos de entrada	Tiempo de ejecución
1	anio= 2022, N = 6	257.70 ms
2	anio= 2024, N = 8	269.72 ms
3	anio= 2023, N = 34	288.55 ms
4	anio= 2021, N = 17	257.11 ms
5	anio= 2020, N = 20	224.56 ms
Promedio de ejecución		259.528 ms

Requerimiento 5

Descripción

```
def req_5(catalog, horsepower, delta, n):  
    """  
    Retorna el resultado del requerimiento 5  
    """  
    # TODO: Modificar el requerimiento 5  
    star_time = get_time()  
    horsepower_min = horsepower - delta  
    horsepower_max = horsepower + delta  
  
    lista_carros = catalog["lista_general"]  
    size = al.size(lista_carros)
```

Este requerimiento identifica los colores de carros más vendidos dentro de un rango de potencia específico. El proceso se realiza haciendo un filtrado sobre el catálogo para agrupar los colores en una Tabla de Hash, realizando el conteo de ventas y calculando los promedios solicitados. Para organizar estos grupos segun el volumen de ventas y poder retornar un Top N sin usar un algoritmo de ordenamiento, se utiliza una Cola de Prioridad. La función retorna un diccionario debido a la cantidad de información que debe dar para facilitar la salida de los datos en el view.

Entrada	Catálogo, horsepower, delta para calcular el rango y n (para el Top n)
Salidas	Diccionario con tiempo de ejecucion, total de carros que cumplen con el filtro y Top N de los colores con carros más vendidos dentro del filtro.
Implementado (Sí/No)	Si, Implementado 50% Danna, 35% Nicolas, 15% Juan

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
-------	-------------

Filtrado inicial y agrupamiento - For sobre catálogo completo (N)	$O(N)$
Agrupamiento por colores - Usando <code>lp.get ()</code> y <code>lp.put ()</code> registrando los colores en lista auxiliar	$O(1)$
For que que recorre lista auxiliar con colores únicos encontrados	$O(C)$, donde C son los colores que tiene los carros
Creación del ranking - Se utiliza una cola de prioridad, <code>pq.insert()</code> por cada uno de los C colores	$O(C \log C)$
Top N - For que ejecuta n veces el <code>pq.remove()</code>	$O(n \log C)$, donde n es el parámetro de entrada
TOTAL	$O(N+C \log C)$

Análisis

El algoritmo es eficiente porque realiza un filtrado y reduce la cantidad de datos que se deben analizar. Aunque primero revisa todo el catalogo N, luego usa un mapa y una sublista de colores para no recorrer toda la tabla de hash, procesadno solo los colores. Finalmente usa un Heap para ordenar el Top N de forma casi instantánea, evitando usar algoritmos de ordenamiento y asegurando que el algoritmo sea escalable con datos grandes.

# de prueba	Datos de entrada	Tiempo de ejecución
1	horsepower = 300 delta = 20 n = 12	228.26 ms
2	horsepower = 400 delta = 30 n = 20	267.36 ms
3	horsepower = 700 delta = 98 n = 15	173.8 ms
4	horsepower = 100 delta = 68 n = 50	175.89 ms
5	horsepower = 350 delta = 10 n = 5	191.34 ms
Promedio de ejecución		207,33 ms

Requerimiento 6

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Para este requerimiento se utilizo una combinacion entre array list, tabla de hash LP y RBTs. Primero, se recorre la lista general de las ventas y se filtran las que cumplen con el rango de años y precios solicitados por parametros. Despues, se agrupan las ventas filtradas en una tabla de hash con la informacion necesaria para calcular el precio promedio, la desviacion estandar, estabilidad y promedio de horsepower. Cada modelo se inserta a un RBT nuevo respetando el orden solicitado: estabilidad ascendente, desviacion estandar ascendente y orden alfabetico del modelo. Se recorren los primeros M elementos ordenados para sacar la respuesta final.

Entrada	Año minimo, año maximo, precio minimo, precio maximo, m (cantidad de modelos a mostrar)
Salidas	Tiempo en milisegundos, numero total de modelos considerados, lista de los modelos mas estables con su: modelo, total de ventas, precio promedio, desviacion estandar, estabilidad, horsepower promedio y informacion de la venta representativa (model, year, fuel type, base price (USD), horsepower y turbo)
Implementado (Sí/No)	Si se implementó 50% Juan, 25% Danna, 25% Nicolas

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Recorrido y filtro de ventas	$O(N)$
Creacion de grupos de ventas por modelo en tabla de hash	$O(N)$
Calculo de características por cada modelo en el filtro	$O(M)$
Busqueda de venta representativa	$O(N)$
Insercion de modelos en RBT	$O(M \log M)$
Sacar solo los primeros M resultados	$O(M)$
TOTAL	$O(N + M \log M)$

Análisis

La eficiencia de este requerimiento se debe al uso de las tablas de hash para agrupar las ventas por modelo, evitando recorridos innecesarios sobre toda la informacion cargada. Solo se recorre la lista general una vez para sacar toda la informacion necesaria. El uso de RBT os permite mantener el orden de manera automatica sin necesidad de algoritmos de ordenamiento adicionales.

Procesador: 1,4 GHz Intel Core i5 de cuatro núcleos

# de prueba	Datos de entrada	Tiempo de ejecución
1	2020, 2025, 40000, 120000, 5	1351.67 ms
2	2019, 2025, 50000, 100000, 5	982.64 ms
3	2023, 2025, 60000, 160000, 7	700.41 ms
4	2020, 2022, 35000, 90000, 5	644.49 ms
5	2021, 2025, 80000, 200000, 10	768.56 ms
Promedio de ejecución		889.55 ms

